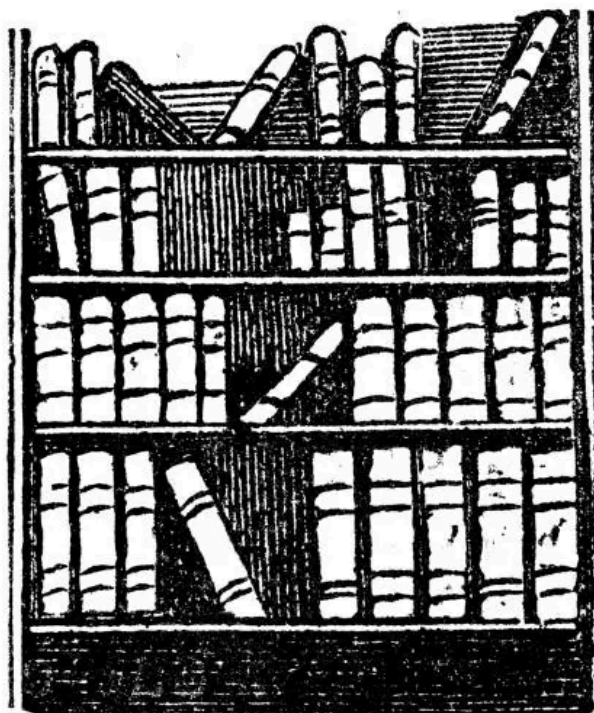


Relatório

1º Trabalho Prático

Sistema Distribuídos



Identificações:

Diogo Tavares (Nº 58049)

Francisco Rodrigues (Nº 59119)

Introdução

→ **Domínio** - Biblioteca, Este projeto implementa um sistema distribuído de gestão de uma biblioteca, onde os clientes podem registar-se, consultar recursos e realizar empréstimos, enquanto o administrador controla o estado dos utilizadores e dos recursos.

O sistema é composto por duas aplicações distintas:

- Cliente Geral (RMI) – utilizado pelos utilizadores da biblioteca para operações como registo, empréstimos e devoluções. Comunica com o servidor por RMI, que utiliza JDBC para acessar à base de dados.
- Cliente Administrador (TCP) – usado pelos funcionários. Comunica com o servidor por sockets TCP e permite aprovar utilizadores e recursos, registar novos itens, e consultar empréstimos ativos.

Esta separação mostra duas formas diferentes de comunicação em sistemas distribuídos, aumenta a modularidade e garante que utilizadores normais não consigam executar ações administrativas.



Criação da Base de Dados e Schema

Para criar a base de dados deve abrir o pgadmin e clicar com o botão direito na Base de Dados PostGres que seria a default do postGresql(Se decidir usar outra base de dados deve modificar o nome no ficheiro config.properties) e clicar em restore...

Restore (Database: postgres)

General Data Options Query Options Table Options Options

Format Custom or tar

Filename

Number of jobs

Role name Select an item...

Please provide a filename.

Close Reset Restore

Depois deve clicar em fileName e seleccionar o ficheiro BaseDados.sql na pasta do trabalho e clicar em restore e assim o schema Biblioteca vai ser criado para estar conforme o trabalho realizado.

Caso prefira usar uma Query SQL, é só copiar o ficheiro Query.txt e colar na query SQL do pgadmin e dar run para criar a schema e as tabelas.



Entidades da base de dados

→ UTILIZADOR:

- ◆ representa qualquer pessoa registrada no sistema (leitor ou administrador).
- ◆ Atributos:
 - **id_utilizador** (PK) – identificador único
 - **nome_completo** – nome completo do utilizador
 - **email** (UNIQUE) – email de contacto e login lógico.
 - **tipo** (ADM | USER) – perfil que indica se terá permissões ou não
 - **estado_conta** (ATIVA | SUSPENSA | PENDENTE) – estado operacional da conta.
 - **estado_admin** (APROVADO | NAO_APROVADO) – validação administrativa para poder operar.

Observação: só utilizadores APROVADOS e com conta ATIVA podem criar empréstimos.

→ Livro:

- ◆ Dados específicos de cada livro(metadados e gestão de cópias).
- ◆ Atributos:
 - **id_recurso** (PK/FK → recurso) – identificador único que liga ao recurso genérico.
 - **titulo** – título do livro.
 - **autor** – autor(es).
 - **num_copias** (≥ 0) – número de cópias disponíveis (decrementa no empréstimo, incrementa na devolução).

Observação: quando num_copias = 0, o recurso.estado fica INDISPONIVEL.



→ **Computador:**

- ◆ Dados relativos aos computadores disponíveis para empréstimo.
- ◆ atributos:
 - **id_recurso** (PK/FK → recurso) – identificador único que liga ao recurso genérico.
 - **modelo** – identificação/modelo do equipamento.
 - **utilizacoes** (≥ 0) – contador de usos (incrementa a cada devolução).

Observações:

Ao iniciar um empréstimo de computador, o recurso.estado passa para EMPRESTADO.

→ **Sala_estudo:**

- ◆ Dados relativos a salas da biblioteca que podem ser reservadas
- ◆ Atributos:
 - **id_recurso** (PK/FK → recurso) – identificador único que liga ao recurso genérico.
 - **nome** – identificação da sala.
 - **capacidade** (> 0) – nº máximo de pessoas permitidas.
 - **localizacao** – nome do polo (default “Colegio Luis Antonio Verney”).

Observações:

Ao criar “empréstimo/reserva” de sala, validar nº de pessoas $\leq$ capacidade e ajustar recurso.estado (ex.: RESERVADA durante o período).

→ **Emprestimo:**

- ◆ regista o empréstimo de um utilizador sobre um recurso num intervalo de tempo.
- ◆ atributos:
 - **id_emprestimo** (PK, SERIAL) – gerado automaticamente (não depende do id_recurso).
 - **id_utilizador** (FK → utilizador) – quem pede.



- **id_recurso** (FK → recurso) – o que é emprestado.
- **data_inicio** (DEFAULT NOW()) – início do empréstimo.
- **data_fim** – fim previsto (início + N dias da duração).
- **estado** (ATIVO | DEVOLVIDO | ATRASADO) – estado do empréstimo.

Observações:

“ATRASADO” quando NOW() > data_fim e ainda não foi devolvido.
Ao devolver: livros ganham 1 cópia e podem voltar a DISPONIVEL;
computadores/salas voltam a DISPONIVEL.

→ Valores possíveis dos tipos ENUM

- ◆ **tipo_utilizador_enum** → ADM / USER
- ◆ **estado_conta_enum** → ATIVA / SUSPENSA / PENDENTE
- ◆ **estado_admin_enum** → APROVADO / NAO_APROVADO
- ◆ **tipo_recurso_enum** → LIVRO / COMPUTADOR / SALA_ESTUDO
- ◆ **estado_recurso_enum** → DISPONIVEL / EMPRESTADO /
INDISPONIVEL / MANUTENCAO / RESERVADA
- ◆ **estado_emprestimo_enum** → ATIVO / DEVOLVIDO / ATRASADO

Funcionalidades

→ **Cliente Geral**

- ◆ **Registar Utilizador** → inicia o processo de criar uma conta no qual pede o nome, o email e o tipo do utilizador. Se o utilizador for do tipo USER ele aguarda aprovação de um administrador para poder fazer empréstimos
- ◆ **Listar Recursos** →
 1. pergunta se quer listar os recursos disponíveis ou indisponíveis
 2. pergunta que tipos de recursos(livros/computadores/sala) queremos listar
 3. é pedido texto para pesquisa entre os recursos



4. os recursos são listados

- ◆ **Listar Empréstimos Ativos** → apresenta os dados dos empréstimos que ainda estão em curso (utilizador que fez o empréstimo; id do recurso emprestado; intervalo de tempo do empréstimo)
- ◆ **Consultar o meu Histórico** → permite ao utilizador ver todos os recursos já emprestados e devoluções.
- ◆ **Pedir Empréstimo** → cria um empréstimo se o utilizador estiver aprovado e o recurso disponível.
- ◆ **Devolver** → termina um empréstimo e aumenta 1 cópia disponível no caso de livros, aumenta 1 utilização no caso dos computadores ou disponibiliza a sala

→ Administrador:

- ◆ **Listar Utilizadores** → **Aprovados/Não Aprovados** mostra utilizadores por estado administrativo.
- ◆ **Aprovar Utilizadores** → autoriza utilizadores a utilizar o sistema.
- ◆ **Histórico de um Utilizador** → permite ver todos os empréstimos de um utilizador específico.
- ◆ **Listar Recursos** → apresenta recursos conforme validação administrativa.
- ◆ **Aprovar Recursos** → Aprova ou “ativa” livros, computadores ou salas para uso público.



- ◆ **Gestão de atributos/recursos** → muda um recurso para disponível, emprestado, indisponível ou manutenção. Altera o N° cópias de um livro ou o n° de usos de um computador
- ◆ **Listar Empréstimos Ativos** → apresenta os empréstimos atualmente ativos
- ◆ **Listar Empréstimos Ativos** → mostra todos os empréstimos que ainda estão em curso.
- ◆ **Registrar Livro/Computador/Sala** → adiciona novos recursos ao sistema.

Problemas e dificuldades resolvidas:

- Problema: Empréstimos criavam `id_emprestimo = id_recurso`
 - ◆ Fix: Passámos a usar SERIAL no PostgreSQL e, no Java, `PreparedStatement.RETURN_GENERATED_KEYS` para obter automaticamente o novo ID gerado evitando assim colisões e permitindo múltiplos empréstimos distintos.
- Problema: Livros podiam ser emprestados com `num_copias = 0`
 - ◆ Fix: Antes de criar o empréstimo, o código faz `SELECT num_copias ... FOR UPDATE` e só continua se `num_copias > 0`. Depois decrementa e atualiza o estado para DISPONIVEL ou INDISPONIVEL. Tudo dentro de transação para garantir consistência.
- Problema: Computadores não aumentavam o campo `utilizacoes` após empréstimo
 - ◆ Fix: Após criar o empréstimo, adicionámos `UPDATE computador SET utilizacoes = utilizacoes + 1 WHERE id_recurso = ?`. Assim o contador sobe sempre que um utilizador usa o recurso.
- Problema: Devolver um recurso não repunha o estado e cópias
 - ◆ Fix: Ao devolver um livro, o código faz `num_copias = num_copias + 1` e atualiza estado para DISPONIVEL. Computadores e salas mudam o estado para DISPONIVEL. Tudo feito dentro de uma única transação.
- Problema: Listagens exibiam recursos sem o nome/título



- ◆ Fix: No SQL passámos a usar COALESCE(livro.titulo, computador.modelo, sala_estudo.nome) para escolher dinamicamente o nome do recurso consoante o tipo.
- Problema: ClienteAdmin enviava comandos diferentes do que o servidor entendia
 - ◆ Fix: Ajustámos para SET_ESTADO_RECURSO ter equivalente SET_ESTADO no servidor, e adicionámos suporte para ambos. Isso eliminou o erro “Comando desconhecido”.
- Problema: Salas de estudo não apareciam ao listar
 - ◆ Fix: Corrigimos o filtro SQL que ignorava tipo_recurso='SALA_ESTUDO'. Agora a query considera livros, computadores e salas, e respeita os filtros de estado e aprovação.
- Problema: Pedido de empréstimo para sala sem validar capacidade
 - ◆ Fix: No criarEmprestimoSalaEstudo(...), o código compara pessoas <= capacidade. Se exceder, o empréstimo é rejeitado com mensagem clara.
- Problema: Empréstimo não mudava estado de computadores/salas
 - ◆ Fix: Após inserir empréstimo, executamos UPDATE recurso SET estado='EMPRESTADO' WHERE id_recurso=?, garantindo coerência visual no sistema.
- Problema: Listagem de aprovados e reprovados misturava resultados
 - ◆ Fix: Criámos SQL distinta para estado_admin='APROVADO' e estado_admin='NAO_APROVADO', com cabeçalhos por tipo de recurso — saída mais clara e fácil de gerir.
- Problema: String aprovado/nao_aprovado falhava por maiúsculas
 - ◆ Fix: Antes de mandar para SQL, convertemos para .toUpperCase(). Agora funciona independentemente do utilizador escrever “Aprovado”, “aprovado”, “APROVADO”